

封装

- 1 定义：
- 2 封装是将数据（属性）和操作数据的方法（行为）捆绑在一起，形成一个独立的“类”（`class`），并通过访问控制来隐藏内部实现的细节。外部代码只能通过类提供的公共接口与对象交互，无法直接访问或修改内部数据。
- 3
- 4 优点：
- 5 安全性：防止数据被意外修改。
- 6 灵活性：内部实现修改不影响外部调用。
- 7 模块化：高内聚、低耦合的代码结构。

继承

- 1 定义：
- 2 继承允许一个类（子类/派生类）基于另一个类（父类/基类）来构建，继承其属性和方法，并可以扩展或修改功能。这实现了代码复用和层次化设计。

多态

- 1 一个接口，多种方法
- 2
- 3 具体体现：编译、运行两个阶段
- 4
- 5 静态多态：重载、模板
- 6 动态多态：虚函数，覆盖

在 Ubuntu 22.04 中，将文件夹打包成 `.deb` 文件需要使用 `dpkg-deb` 工具。以下是一个详细的步骤指南，帮助你将一个文件夹打包成 `.deb` 文件：

打包编译deb

1. 创建必要的目录结构：

你需要创建一个标准的 Debian 包目录结构。通常，这个目录结构如下：

```
1 your-package/  
2 |— DEBIAN  
3 |   └─ control  
4 └─ opt  
5     └─ your-directory
```

- `DEBIAN`：包含控制文件 `control`。
- `opt`：包含你想要打包的文件夹。

2. 创建控制文件：

在 `DEBIAN` 目录下创建一个 `control` 文件，内容如下：

```
1 Package: your-package-name
2 Version: 1.0
3 Section: base
4 Priority: optional
5 Architecture: all
6 Depends:
7 Maintainer: Your Name <your.email@example.com>
8 Description: A brief description of your package
```

- `Package`：包的名称。
- `Version`：包的版本。
- `Section`：包的分类。
- `Priority`：包的优先级。
- `Architecture`：目标架构，通常为 `all` 表示所有架构都适用。
- `Depends`：依赖的其他软件包，如果有依赖项，用逗号分隔列表。
- `Maintainer`：包的维护者信息。
- `Description`：包的简要描述。

示例

假设你要打包的文件夹名称为 `my-app`，并且包含一些文件和子目录。

1. 创建目录结构：

```
1 mkdir -p my-app/DEBIAN
2 mkdir -p my-app/opt/my-app/
```

2. 复制文件：

```
1 cp -r /path/to/your-files my-app/opt/my-app/
```

3. 创建控制文件：

```
1 nano my-app/DEBIAN/control
```

填写控制文件内容，例如：

```
1 Package: my-app
2 Version: 1.0
3 Section: base
4 Priority: optional
5 Architecture: all
6 Maintainer: Your Name <your.email@example.com>
7 Description: My application
```

4. 打包成 .deb 文件：

使用 `dpkg-deb` 命令将目录结构打包成 `.deb` 文件：

```
1 | dpkg-deb --build my-app
```

这将生成一个名为 `my-app.deb` 的文件。

安装 .deb 文件

你可以使用以下命令安装生成的 `.deb` 文件：

```
1 | sudo dpkg -i my-app.deb
```

验证安装

安装完成后，你可以验证文件是否正确安装到 `/opt/my-app` 目录：

```
1 | ls /opt/my-app
```

清理

如果你需要删除生成的 `.deb` 文件和临时目录，可以使用以下命令：

```
1 | rm my-app.deb
2 | rm -r my-app
```

通过以上步骤，你应该能够成功将一个文件夹打包成 `.deb` 文件，并在 Ubuntu 22.04 上安装和使用它。

什么是指针数组和数组指针

指针数组是一个数组，其中的每个元素都是指针。这些指针可以指向不同的内存地址，通常用于存储一组相同类型的指针。

```
// ptrArray 是一个包含 5 个元素的数组，
// 每个元素都是 int* 类型的指针，可以分别指向不同的整数
```

```
int *ptrArray[5];
```

数组指针是一个指针，它指向数组的首地址。它本身是一个指针，但指向的内容是一个数组对象。

什么是函数指针和指针函数以及区别

函数指针 是指 指向一个函数的指针变量

指针函数 指的是 返回类型 为 指向函数的指针 的函数。换句话说，指针函数是一个返回类型为函数指针的函数。

什么是常量指针和指针常量以及区别

C++的引用和 C 语言的指针有什么区别

指针和引用主要有以下区别：

- (1) 引用必须被初始化，但是不分配存储空间。指针不声明时初始化，在初始化的时候需要分配存储空间。
- (2) 引用初始化以后不能被改变，指针可以改变所指的對象。
- (3) 不存在指向空值的引用，但是存在指向空值的指针。

链表和数组有什么区别

数组和链表有以下几点不同：

- (1) 存储形式：数组是一块连续的空间，声明时就要确定长度。链表是一块可不连续的动态空间，长度可变，每个结点要保存相邻结点指针。
- (2) 数据查找：数组的线性查找速度快，查找操作直接使用偏移地址。链表需要按顺序检索结点，效率低。
- (3) 数据插入或删除：链表可以快速插入和删除结点，而数组则可能需要大量数据移动。
- (4) 越界问题：链表不存在越界问题，数组有越界问题。

说明：在选择数组或链表数据结构时，一定要根据实际需要进行选择。数组便于查询，链表便于插入删除。数组节省空间但是长度固定，链表虽然变长但是占了更多的存储空间。

简述队列和栈的异同

队列和栈都是线性存储结构，但是两者的插入和删除数据的操作不同，队列是“先进先出”，栈是“后进先出”。

注意：区别栈区和堆区。堆区的存取是“顺序随意”，而栈区是“后进先出”。栈由编译器自动分配释放，存放函数的参数值，局部变量的值等。其操作方式类似于数据结构中的栈。堆一般由程序员分配释放，若程序员不释放，程序结束时可能由 OS 回收。分配方式类似于链表。

它与本题中的堆和栈是两回事。堆栈只是一种数据结构，而堆区和栈区是程序的不同内存存储区域。

简述类成员函数的重写、重载和隐藏的区别

(1) 重写和重载主要有以下几点不同。

- 范围的区别：被重写的和重写的函数在两个类中，而重载和被重载的函数在同一个类中。
- 参数的区别：被重写函数和重写函数的参数列表一定相同，而被重载函数和重载函数的参数列表一定不同。
- virtual 的区别：重写的基类中被重写的函数必须要有 virtual 修饰，而重载函数和被重载函数可以被 virtual 修饰，也可以没有。

(2) 隐藏和重写、重载有以下几点不同。

- 与重载的范围不同：和重写一样，隐藏函数和被隐藏函数不在同一个类中。
- 参数的区别：隐藏函数和被隐藏的函数的参数列表可以相同，也可不同，但是函数名肯定要相同。当参数不相同，无论基类中的参数是否被 virtual 修饰，基类的函数都是被隐藏，而不是被重写。

说明：虽然重载和覆盖都是实现多态的基础，但是两者实现的技术完全不相同，达到的目的也是完全不同的，覆盖是动态态绑定的多态，而重载是静态绑定的多态。

1. 重载

- 在同一个类中，方法名相同但参数列表不同。
- 用于提供多种方法实现。

2. 重写

- 子类重新定义父类的虚方法。
- 方法名、参数列表和返回值类型必须相同。

- 用于实现多态。

3. 覆盖

- 子类重新定义父类的非虚方法。
- 方法名和参数列表相同，但返回值类型可以不同。
- 不会影响多态行为。

C 语言的关键字 static 和 C++ 的关键字 static 有什么区别

在 C 中 static 用来修饰局部静态变量和外部静态变量、函数。而 C++ 中除了上述功能外，还用来定义类的成员变量和函数。即静态成员和静态成员函数。

注意：编程时 static 的记忆性，和全局性的特点可以让在不同时期调用的函数进行通信，传递信息，而 C++ 的静态成员则可以在多个对象实例间进行通信，传递信息。

C++ 的空类有哪些成员函数

- 缺省构造函数。
- 缺省拷贝构造函数。
- 缺省析构函数。
- 缺省赋值运算符。
- 缺省取址运算符。
- 缺省取址运算符 const。

注意：有些书上只是简单的介绍了前四个函数。没有提及后面这两个函数。但后面这两个函数也是空类的默认函数。另外需要注意的是，只有当实际使用这些函数的时候，编译器才会去定义它们。

谈谈你对拷贝构造函数和赋值运算符的认识

拷贝构造函数和赋值运算符重载有以下两个不同之处：

- (1) 拷贝构造函数生成新的类对象，而赋值运算符不能。
- (2) 由于拷贝构造函数是直接构造一个新的类对象，所以在初始化这个对象之前不用检验源对象是否和新建对象相同。而赋值运算符则需要这个操作，另外赋值运算中如果原来的对象中有内存分配要先把内存释放掉

注意：当有类中有指针类型的成员变量时，一定要重写拷贝构造函数和赋值运算符，不要使用默认的。

面向对象的三大特征

面向对象的三大特征是封装性、继承性和多态性：

- 封装性：将客观事物抽象成类，每个类对自身的数据和方法实行 protection (private, protected, public) 。
- 继承性：广义的继承有三种实现形式：实现继承（使用基类的属性和方法而无需额外编码的能力）、可视继承(子窗体使用父窗体的外观和实现代码)、接口继承(仅使用属性和方法,实现滞后到子类实现)。
- 多态性：是将父类对象设置成为和一个或多个它的子对象相等的技术。用子类对象给父类对象赋值之后，父类对象就可以根据当前赋值给它的子对象的特性以不同的方式运作。这部分需要熟悉掌握原理虚函数，了解一些概念（静态多态、动态多态）等，面试时经常会问。

说明：面向对象的三个特征是实现面向对象技术的关键，每一个特征的相关技术都非常的复杂，程序员应该多看、多练。

进程线程

- 1 # 堆栈段 (stack) :
2 由编译器自动分配和释放，存放函数的参数和局部变量。操作方式类似于数据结构中的栈。
- 3 # 堆 (heap) :

4 由程序员分配和释放。如果程序员没有释放，会造成内存泄漏堆得操作方式和数据结构中的堆不同，堆空间的操作方式类似于链表。

5 **# BSS:**

6 存放未初始化的全局变量和**static**静态变量。

7 **# 数据段 (data):**

8 存放初始化之后的全局变量和**static**静态变量

9 **# 代码段:**

10 以二进制格式存放程序和函数主体。

11

12 **# 进程和线程相关**

13 进程有独立的地址空间，是资源分配的最小单位。线程没有独立的地址空间，是系统调度的最小单位（执行运算的最小单位）。

14 一个程序最少有一个进程，一个进程最少有一个线程。

15 线程的崩溃基本上会导致进程的崩溃，所以多进程比多线程的程序健壮性更好，多线程开销小，效率高，但是不便于资源的保护和管理。多进程相反

16 进程有独立的内存，多线程共享这个进程的内存，所以多线程编程极大的提高了程序的运行效率，在多核CPU的系统中多线程可以充分利用CPU多核运算的能力。

17

18 (1) 创建态：系统完成创建进程的一系列工作。只能转换到就绪态。

19

20 (2) 就绪态：拥有除过CPU之外的其他所需的所有资源。当拥有CPU时就可以转换到运行态。

21

22 (3) 运行态：用于CPU和所需的所有资源。

23

24 当时间片到或者处理机被抢占了，就转换到就绪态；

25

26 当进程用【系统调用】的方式申请某种系统资源或者请求等待某个事件的发生，则进入阻塞态。（主动）

27

28 (4) 阻塞态：没有所需要的资源。当所需要的资源得到分配时，进入就绪态（被动）。

29

30 (5) 终止态：进程运行结束或者于大盘不可修复的错误时，由运行态转到终止态。

31

32

33 创建守护进程

34

35 调用**fork()**，创建新进程，它会是将来的守护进程。

36 在父进程中调用**exit**，保证子进程不是进程组长

37 调用**setsid()**创建新的会话区，子进程成为会话首进程

38 将当前目录改成根目录(如果把当前目录作为守护进程的目录，当前目录不能被卸载他作为守护进程的工作目录)，关闭文件描述符

39 忽略信号，进行后台服务逻辑处理

1 进程：指在系统中正在运行的一个应用程序；程序一旦运行就是进程；进程——资源分配的最小单位。

1 线程：系统分配处理器时间资源的基本单元，或者说进程之内独立执行的一个单元执行流。是程序执行的最小单位。

- 1 对于操作系统来说，一个任务就是一个进程（**Process**），比如打开一个浏览器就是启动一个浏览器进程，打开一个记事本就启动了一个记事本进程，打开两个记事本就启动了两个记事本进程，打开一个word就启动了一个word进程。
- 2
- 3 有些进程还不止同时干一件事，比如**word**，它可以同时进行打字、拼写检查、打印等事情。在一个进程内部，要同时干多件事，就需要同时运行多个“子任务”，我们把进程内的这些“子任务”称为线程（**Thread**）。

一、Linux内核架构

1. Linux内核由哪些主要子系统组成？

答案：

- 进程调度子系统（**SCHED**）
- 内存管理子系统（**MM**）
- 虚拟文件系统（**VFS**）
- 网络子系统（**NET**）
- 进程间通信（**IPC**）
- 设备驱动和硬件抽象层

2. 用户空间和内核空间的区别？

答案：

- **用户空间**：应用程序运行的非特权模式，无法直接访问硬件。
- **内核空间**：操作系统核心代码运行的特权模式，可直接操作硬件。

3. `proc` 和 `sysfs` 的作用？

答案：

- **/proc**：虚拟文件系统，提供进程和内核信息的接口（如 `/proc/cpuinfo`）。
- **/sys**：导出设备、驱动和内核对象的层次化视图，用于配置硬件。

4. 系统调用（`syscall`）的流程？

答案：

1. 用户程序触发软中断（如 `int 0x80` 或 `syscall` 指令）。
2. 切换到内核模式，执行对应的系统调用处理函数。
3. 结果返回用户空间。

5. 内核同步机制有哪些？

答案：

- **自旋锁**（`spinlock`）：忙等待，适合短期锁定。
 - **信号量**（`semaphore`）：睡眠等待，适合长期锁定。
 - **RCU**（`Read-Copy-Update`）：无锁读，写时复制。
-

二、文件系统

文件系统是操作系统管理磁盘数据的机制

1. VFS（虚拟文件系统）的作用？

答案：

抽象不同文件系统的差异，为上层提供统一的接口（如 `open()`、`read()`）。

2. inode和dentry的区别？

答案：

- **inode**：存储文件的元数据（权限、大小、块位置等）。
- **dentry**：目录项，缓存目录结构，加速路径查找。

3. ext4、XFS、Btrfs的区别？

答案：

- **ext4**：默认文件系统，支持日志，最大16TB文件。
- **XFS**：适合大文件和高并发，支持动态inode分配。
- **Btrfs**：支持写时复制（CoW）、快照、RAID。

4. 硬链接和软链接的区别？

答案：

- **硬链接**：指向同一inode，不能跨文件系统，删除原文件不影响链接。
- **软链接**：独立inode，存储目标路径，可跨文件系统，原文件删除后失效。

5. 文件系统日志（Journaling）的作用？

答案：

记录文件系统操作日志，确保崩溃后能快速恢复一致性。

文件系统的核心操作

(1) 文件读写流程

1. 打开文件：

- 通过路径解析找到文件的inode。
- 内核创建**文件描述符**（fd），关联到进程的打开文件表。

2. 读取文件：

- 根据inode中的块指针，从磁盘读取数据块。
- 数据可能被缓存到**页缓存（Page Cache）**中加速后续访问。

3. 写入文件：

- 数据先写入页缓存，由内核异步刷盘。
- 若文件系统启用**日志（Journaling）**，写入前会记录日志以确保崩溃恢复。

(2) 文件系统挂载

- **挂载（Mount）**：将磁盘分区关联到目录树的一个节点（如将 `/dev/sda1` 挂载到 `/mnt`）。
 - **虚拟文件系统（VFS）**：抽象层，允许不同文件系统（ext4、NTFS、FAT）以统一接口接入内核。
-

三、内存管理

1. 分页 (Paging) 和分段 (Segmentation) 的区别?

答案:

- **分页**: 物理内存划分为固定大小的页, 解决外部碎片。
- **分段**: 按逻辑单元 (代码、数据) 划分, 长度可变, 易产生外部碎片。

2. 物理地址和虚拟地址如何转换?

答案:

通过MMU (内存管理单元) 和页表 (Page Table) 完成映射。

3. 伙伴系统 (Buddy System) 的作用?

答案:

管理物理内存页, 减少外部碎片, 通过合并/拆分 2^n 大小的块分配内存。

4. 什么是页缓存 (Page Cache) ?

答案:

内核将磁盘文件缓存在内存中, 加速文件读写 (通过 `read()` / `write()`) 。

5. OOM Killer如何选择终止进程?

答案:

根据进程的 `oom_score` (内存占用、运行时间、优先级等) 选择最“不重要”的进程终止。

四、进程调度

1. 进程和线程的区别?

答案:

- **进程**: 资源分配单位, 有独立地址空间。
- **线程**: CPU调度单位, 共享进程资源。

2. CFS (完全公平调度器) 如何工作?

答案:

基于虚拟时间 (vruntime) 的红黑树选择最小vruntime的进程, 确保公平性。

3. 实时进程的调度策略?

答案:

- **SCHED_FIFO**: 先进先出, 直到主动让出CPU。
- **SCHED_RR**: 时间片轮转, 优先级相同则轮流执行。

4. 什么是上下文切换 (Context Switch) ?

答案:

保存当前进程状态 (寄存器、页表等), 恢复下一个进程状态。

5. 僵尸进程 (Zombie) 的产生和解决?

答案:

- **原因**: 子进程退出后父进程未调用 `wait()` 回收状态。
 - **解决**: 终止父进程或强制回收 (`kill` 父进程) 。
-

五、其他经典问题

1. 如何查看进程的CPU和内存使用？

`top`、`htop`、`ps aux`。

2. `chmod 755` 和 `chmod u+rw,g+rx,o+rx` 的区别？

两者等价，755表示权限位 `rw-r-xr-x`。

3. 进程间通信（IPC）方式？

管道、消息队列、共享内存、信号量、Socket。

4. 解释Copy-on-Write（写时复制）

父子进程共享内存页，仅当修改时复制新页，减少开销

进程调度

进程调度主要有两种方式：**非剥夺调度**和**剥夺调度**¹。

非剥夺调度：当一个进程正在处理机上执行时，即使有更为重要或紧迫的进程进入就绪队列，仍然让正在执行的进程继续执行，直到该进程完成或进入阻塞态时，才把处理机分配给更为重要的进程。其优点是实现简单，系统开销小，适用于大多数批处理系统²。

剥夺调度：当一个进程正在处理机上执行时，若有某个更为重要或紧迫的进程需要使用处理机，则立即暂停正在执行的进程，将处理机分配给这个更重要的进程。这种方式对提高系统吞吐率和响应效率都有明显的好处²。

调度算法

常见的调度算法包括：

先来先服务（FCFS）：按照进程到达的顺序进行调度，适用于批处理系统¹。**短作业优先（SJF）**：优先调度估计运行时间最短的进程，但可能导致长作业饿死¹。**优先级调度**：根据进程的优先级进行调度，优先级可以是静态的或动态的¹。**时间片轮转**：将所有就绪进程按到达时间的先后次序排成一个队列，每个进程只能运行一个时间片，适用于分时系统¹。**多级反馈队列**：结合时间片轮转和优先级调度，通过动态调整进程优先级和时间片的大小，兼顾系统的多方面目标¹。

调度准则

为了比较处理机调度算法的性能，常用的评价准则包括¹：

CPU利用率：尽可能使得CPU保持忙的状态，资源利用率尽可能高。**系统吞吐量**：单位时间内CPU完成的作业数量。**周转时间**：从作业提交到作业完成所经历的时间。**等待时间**：进程处于等处理机状态的时间之和。**响应时间**：用户提交请求到系统首次产生响应所用的时间。

strace

- 1) 调试程序
- 2) 故障、排除程序
- 3) 拦截进程的系统调用
- 4) 记录进程的系统调用
- 5) 进程接收到的信号
- 6) 跟踪正在运行的进程

启动进程并跟踪

可以通过 `strace` 启动要跟踪的进程。例如：

```
1 | $ strace ls -lh /var/log/messages
```

跟踪已运行的进程

使用 `-p` 选项指定进程 ID (pid) 来跟踪已运行的进程：

pidof some_server

strace -p 17553

按 `Ctrl + C` 结束跟踪¹。

常用选项

`-tt`：显示毫秒级别的时间戳。`-T`：显示每次系统调用所花费的时间。`-v`：显示详细信息，如环境变量、文件 stat 结构等。`-f`：跟踪目标进程及其子进程。`-e trace=file`：仅跟踪与文件访问相关的系统调用。`-o`：将输出写入指定文件。`-s`：指定字符串参数的最大输出长度，默认32字节¹。

示例

跟踪 nginx 启动时访问的文件

```
1 | $ strace -tt -T -f -e trace=file -o /data/log/strace.log -s 1024 ./nginx
```

输出中，第一列显示进程的 pid，接着是时间戳，最后一列显示系统调用所花费的时间¹。

定位进程异常退出

假设进程 `run.sh` 异常退出，通过 `ps` 命令获取其 pid，然后使用 `strace` 跟踪：

```
1 | $ strace -o strace.log -tt -p 24298
```

查看 `strace.log`，可以看到进程是被 `SIGKILL` 信号杀死的¹。

定位共享内存异常

假设服务启动时报错，通过 `strace` 跟踪共享内存相关的系统调用：

```
1 | $ strace -tt -f -e trace=ipc ./a_mon_svr ../conf/a_mon_svr.conf
```

进程间通信

- 1 # 无名管道**pipe**: 管道是一种半双工的通信方式, 数据只能单向流动, 而且只能在具有亲缘关系的进程间使用。进程的亲缘关系通常是指父子进程关系。
- 2 # 命名管道**FIFO**: 有名管道也是半双工的通信方式, 但是它允许无亲缘关系进程间的通信。
- 3 # 消息队列**MessageQueue**: 消息队列是由消息的链表, 存放在内核中并由消息队列标识符标识。//消息队列克服了信号传递信息少、管道只能承载无格式字节流以及缓冲区大小受限 等缺点。
- 4 # 共享存储**SharedMemory**: 共享内存就是映射一段能被其他进程所访问的内存, 这段共享内存由一个进程创建, 但多个进程都可以访问。共享内存是最快的 **IPC** 方式, 它是针对 其他进程间通信方式运行效率低而专门设计的。它往往与其他通信机制, 如信号量, 配合使用, 来实现进程间的同步和通信。
- 5 # 信号量**Semaphore**: 信号量是一个计数器, 可以用来控制多个进程对共享资源的访问。它常作为一种锁机制, 防止某进程正在访问共享资源时, 其他进程也访问该资源。因此, 主 要作为进程间以及同一进程内不同线程之间的同步手段。
- 6 # 套接字**Socket**: 套接口也是一种进程间通信机制, 与其他通信机制不同的是, 它可用于不同及其间的进程通信。
- 7 # 信号 (**signal**) : 信号是一种比较复杂的通信方式, 用于通知接收进程某个事件已经发生。

管道

无名管道

```
1 # 无名管道
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <unistd.h>
6 #include <string.h>
7
8 int main() {
9     int pipefd[2]; // 用于存储管道的读端和写端文件描述符
10    pid_t pid;
11
12    // 创建管道
13    if (pipe(pipefd) == -1) {
14        perror("pipe");
15        exit(EXIT_FAILURE);
16    }
17
18    // 创建子进程
19    pid = fork();
20
21    if (pid == -1) {
22        perror("fork");
23        exit(EXIT_FAILURE);
24    }
25
26    if (pid == 0) {
27        // 子进程: 关闭读端, 只负责写数据
28        close(pipefd[0]);
29
30        // 写入数据到管道
31        const char *message = "Hello from child process";
32        write(pipefd[1], message, strlen(message) + 1); // +1 包括 '\0'
33    }
```

```

34     // 关闭写端
35     close(pipefd[1]);
36     exit(EXIT_SUCCESS);
37 } else {
38     // 父进程：关闭写端，只负责读数据
39     close(pipefd[1]);
40
41     // 从管道读取数据
42     char buffer[100];
43     read(pipefd[0], buffer, sizeof(buffer));
44
45     // 输出读取到的数据
46     printf("Parent process received: %s\n", buffer);
47
48     // 关闭读端
49     close(pipefd[0]);
50     wait(NULL); // 等待子进程结束
51 }
52
53 return 0;
54 }
55
56 如果读端没有数据可读，read() 会阻塞，直到写端写入数据。
57 如果写端没有关闭，读端会一直等待。

```

代码说明

1. 创建管道

- 使用 `pipe(pipefd)` 创建一个无名管道，`pipefd` 是一个大小为 2 的整型数组：
 - `pipefd[0]`：管道的读端。
 - `pipefd[1]`：管道的写端。

2. 创建子进程

- 使用 `fork()` 创建一个子进程。
- 父进程和子进程通过管道进行通信。

3. 子进程逻辑

- 子进程关闭读端（`pipefd[0]`），因为子进程只负责写数据。
- 使用 `write(pipefd[1], message, length)` 将数据写入管道。
- 写完后关闭写端（`pipefd[1]`）。

4. 父进程逻辑

- 父进程关闭写端（`pipefd[1]`），因为父进程只负责读数据。
- 使用 `read(pipefd[0], buffer, size)` 从管道中读取数据。
- 输出读取到的数据。
- 关闭读端（`pipefd[0]`），并等待子进程结束。

注意事项

1. 管道的单向性

- 无名管道是单向的，数据只能从写端流向读端。
- 如果需要双向通信，可以创建两个管道。

2. 管道的阻塞行为

- 如果读端没有数据可读，`read()` 会阻塞，直到写端写入数据。
- 如果写端没有关闭，读端会一直等待。

3. 进程间通信的限制

- 无名管道只能用于具有亲缘关系的进程（如父子进程）之间的通信。
- 如果需要在无亲缘关系的进程之间通信，可以使用有名管道（FIFO）。

有名管道

```
1  # 有名管道
2
3  #####写入进程#####
4  #include <stdio.h>
5  #include <stdlib.h>
6  #include <fcntl.h>
7  #include <unistd.h>
8  #include <string.h>
9
10 int main() {
11     int fd;
12     const char *message = "Hello from writing process";
13
14     // 打开有名管道的写端
15     fd = open("my_fifo", O_WRONLY);
16     if (fd == -1) {
17         perror("open");
18         exit(EXIT_FAILURE);
19     }
20
21     // 写入数据
22     write(fd, message, strlen(message) + 1); // +1 包括 '\0'
23
24     // 关闭文件描述符
25     close(fd);
26
27     return 0;
28 }
29
30 #####读取进程#####
31 #include <stdio.h>
32 #include <stdlib.h>
33 #include <fcntl.h>
34 #include <unistd.h>
35 #include <string.h>
36
37 int main() {
38     int fd;
```

```

39     char buffer[100];
40
41     // 打开有名管道的读端
42     fd = open("my_fifo", O_RDONLY);
43     if (fd == -1) {
44         perror("open");
45         exit(EXIT_FAILURE);
46     }
47
48     // 读取数据
49     read(fd, buffer, sizeof(buffer));
50
51     // 输出读取到的数据
52     printf("Reading process received: %s\n", buffer);
53
54     // 关闭文件描述符
55     close(fd);
56
57     return 0;
58 }

```

代码说明

1. 创建有名管道

- 使用 `mkfifo()` 函数创建有名管道，`my_fifo` 是管道的名称。
- 权限 `0666` 表示所有用户都可以读写。

2. 写入进程

- 打开有名管道的写端（`O_WRONLY`）。
- 使用 `write()` 将数据写入管道。
- 写完后关闭文件描述符。

3. 读取进程

- 打开有名管道的读端（`O_RDONLY`）。
- 使用 `read()` 从管道中读取数据。
- 输出读取到的数据。
- 读完后关闭文件描述符。

注意事项

1. 管道的单向性

- 有名管道是单向的，数据只能从写端流向读端。
- 如果需要双向通信，可以创建两个有名管道。

2. 管道的阻塞行为

- 如果读端没有打开，写端会阻塞在 `open()` 调用上。
- 如果写端没有打开，读端会阻塞在 `open()` 调用上。
- 数据写入后，读端会自动解阻塞。

3. 管道的持久性

- 有名管道在文件系统中是一个特殊文件，即使没有进程使用它，它也会存在。

- 使用 `unlink("my_fifo")` 可以删除有名管道。

消息队列

消息队列案例：两个独立进程通信

1. 创建消息队列

- 使用 `msgget()` 创建消息队列。

2. 发送消息

- 使用 `msgsnd()` 将消息发送到消息队列。

3. 接收消息

- 使用 `msgrcv()` 从消息队列中接收消息。

4. 删除消息队列

- 使用 `msgctl()` 删除消息队列。

完整代码

1. 消息结构体

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/ipc.h>
5  #include <sys/msg.h>
6
7  // 消息结构体
8  struct msg_buffer {
9      long mtype; // 消息类型
10     char mtext[100]; // 消息内容
11 };
```

2. 创建消息队列

```
1  int main() {
2      key_t key;
3      int msgid;
4
5      // 创建一个唯一的键
6      key = ftok("progfile", 65);
7      if (key == -1) {
8          perror("ftok");
9          exit(EXIT_FAILURE);
10     }
11
12     // 创建消息队列
13     msgid = msgget(key, 0666 | IPC_CREAT);
14     if (msgid == -1) {
15         perror("msgget");
16         exit(EXIT_FAILURE);
17     }
18
19     return msgid;
```


3. 发送消息

```
1  int main() {
2      key_t key;
3      int msgid;
4      struct msg_buffer msg;
5
6      // 创建一个唯一的键
7      key = ftok("progfile", 65);
8      if (key == -1) {
9          perror("ftok");
10         exit(EXIT_FAILURE);
11     }
12
13     // 获取消息队列
14     msgid = msgget(key, 0666 | IPC_CREAT);
15     if (msgid == -1) {
16         perror("msgget");
17         exit(EXIT_FAILURE);
18     }
19
20     // 准备消息
21     msg.mtype = 1; // 消息类型
22     strcpy(msg.mtext, "Hello from sending process");
23
24     // 发送消息
25     if (msgsnd(msgid, &msg, sizeof(msg), 0) == -1) {
26         perror("msgsnd");
27         exit(EXIT_FAILURE);
28     }
29
30     printf("Message sent: %s\n", msg.mtext);
31
32     return 0;
33 }
```

4. 接收消息

```
1  int main() {
2      key_t key;
3      int msgid;
4      struct msg_buffer msg;
5
6      // 创建一个唯一的键
7      key = ftok("progfile", 65);
8      if (key == -1) {
9          perror("ftok");
10         exit(EXIT_FAILURE);
11     }
12
13     // 获取消息队列
14     msgid = msgget(key, 0666 | IPC_CREAT);
```

```

15     if (msgid == -1) {
16         perror("msgget");
17         exit(EXIT_FAILURE);
18     }
19
20     // 接收消息
21     if (msgrcv(msgid, &msg, sizeof(msg), 1, 0) == -1) {
22         perror("msgrcv");
23         exit(EXIT_FAILURE);
24     }
25
26     // 输出接收到的消息
27     printf("Message received: %s\n", msg.mtext);
28
29     // 删除消息队列
30     if (msgctl(msgid, IPC_RMID, NULL) == -1) {
31         perror("msgctl");
32         exit(EXIT_FAILURE);
33     }
34
35     return 0;
36 }

```

代码说明

1. 创建消息队列

- 使用 `ftok()` 生成一个唯一的键。
- 使用 `msgget()` 创建或获取消息队列，`IPC_CREAT` 标志表示如果消息队列不存在则创建。

2. 发送消息

- 准备一个 `msg_buffer` 结构体，设置消息类型和内容。
- 使用 `msgsnd()` 将消息发送到消息队列。

3. 接收消息

- 使用 `msgrcv()` 从消息队列中接收消息，指定消息类型。
- 输出接收到的消息。
- 使用 `msgctl()` 删除消息队列。

注意事项

1. 消息类型

- 消息类型是一个正整数，用于区分不同类型的消息。
- 接收方可以根据类型选择性地接收消息。

2. 消息大小

- 消息大小不能超过系统定义的最大值（通常为 8192 字节）。

3. 消息队列的持久性

- 消息队列在内核中存在，即使发送方和接收方都退出，消息队列仍然存在。
- 使用 `msgctl()` 删除消息队列。

4. 错误处理

- 在实际应用中，需要仔细处理可能的错误，例如权限问题、消息队列满等。

共享内存

1. 共享内存结构体

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include <sys/ipc.h>
5 #include <sys/shm.h>
6 #include <unistd.h>
7 #include <semaphore.h>
8
9 // 共享内存结构体
10 struct shared_memory {
11     int counter; // 用于同步的计数器
12     char buffer[100]; // 数据缓冲区
13     sem_t mutex; // 用于同步的信号量
14 };
```

2. 创建共享内存

```
1 int main() {
2     key_t key;
3     int shmid;
4     struct shared_memory *shared;
5
6     // 创建一个唯一的键
7     key = ftok("progfile", 65);
8     if (key == -1) {
9         perror("ftok");
10        exit(EXIT_FAILURE);
11    }
12
13    // 创建共享内存
14    shmid = shmget(key, sizeof(struct shared_memory), 0666 | IPC_CREAT);
15    if (shmid == -1) {
16        perror("shmget");
17        exit(EXIT_FAILURE);
18    }
19
20    // 将共享内存附加到当前进程的地址空间
21    shared = (struct shared_memory *)shmat(shmid, (void *)0, 0);
22    if (shared == (struct shared_memory *)(-1)) {
23        perror("shmat");
24        exit(EXIT_FAILURE);
25    }
26
27    // 初始化共享内存
28    shared->counter = 0;
29    sem_init(&shared->mutex, 1, 1); // 初始化信号量
30
31    return shmid;
```

3. 写入共享内存

```

1  int main() {
2      key_t key;
3      int shmid;
4      struct shared_memory *shared;
5
6      // 创建一个唯一的键
7      key = ftok("progfile", 65);
8      if (key == -1) {
9          perror("ftok");
10         exit(EXIT_FAILURE);
11     }
12
13     // 获取共享内存
14     shmid = shmget(key, sizeof(struct shared_memory), 0666 | IPC_CREAT);
15     if (shmid == -1) {
16         perror("shmget");
17         exit(EXIT_FAILURE);
18     }
19
20     // 将共享内存附加到当前进程的地址空间
21     shared = (struct shared_memory *)shmat(shmid, (void *)0, 0);
22     if (shared == (struct shared_memory *)(-1)) {
23         perror("shmat");
24         exit(EXIT_FAILURE);
25     }
26
27     // 写入数据
28     sem_wait(&shared->mutex); // 等待信号量
29     strcpy(shared->buffer, "Hello from writing process");
30     shared->counter++;
31     printf("Message sent: %s\n", shared->buffer);
32     sem_post(&shared->mutex); // 释放信号量
33
34     return 0;
35 }

```

4. 读取共享内存

```

1  int main() {
2      key_t key;
3      int shmid;
4      struct shared_memory *shared;
5
6      // 创建一个唯一的键
7      key = ftok("progfile", 65);
8      if (key == -1) {
9          perror("ftok");
10         exit(EXIT_FAILURE);
11     }
12

```

```

13 // 获取共享内存
14 shm_id = shmget(key, sizeof(struct shared_memory), 0666 | IPC_CREAT);
15 if (shm_id == -1) {
16     perror("shmget");
17     exit(EXIT_FAILURE);
18 }
19
20 // 将共享内存附加到当前进程的地址空间
21 shared = (struct shared_memory *)shmat(shm_id, (void *)0, 0);
22 if (shared == (struct shared_memory *)(-1)) {
23     perror("shmat");
24     exit(EXIT_FAILURE);
25 }
26
27 // 读取数据
28 sem_wait(&shared->mutex); // 等待信号量
29 if (shared->counter > 0) {
30     printf("Message received: %s\n", shared->buffer);
31     shared->counter--;
32 } else {
33     printf("No message available\n");
34 }
35 sem_post(&shared->mutex); // 释放信号量
36
37 // 从进程的地址空间分离共享内存
38 if (shmdt(shared) == -1) {
39     perror("shmdt");
40     exit(EXIT_FAILURE);
41 }
42
43 // 删除共享内存
44 if (shmctl(shm_id, IPC_RMID, NULL) == -1) {
45     perror("shmctl");
46     exit(EXIT_FAILURE);
47 }
48
49 return 0;
50 }

```

代码说明

1. 创建共享内存

- 使用 `ftok()` 生成一个唯一的键。
- 使用 `shmget()` 创建或获取共享内存段。
- 使用 `shmat()` 将共享内存段附加到当前进程的地址空间。
- 初始化共享内存中的信号量。

2. 写入共享内存

- 使用 `sem_wait()` 等待信号量，确保写操作的原子性。
- 将数据写入共享内存。
- 更新计数器。
- 使用 `sem_post()` 释放信号量。

3. 读取共享内存

- 使用 `sem_wait()` 等待信号量，确保读操作的原子性。
- 从共享内存中读取数据。
- 更新计数器。
- 使用 `sem_post()` 释放信号量。
- 使用 `shmdt()` 将共享内存从进程的地址空间分离。
- 使用 `shmctl()` 删除共享内存段。

注意事项

1. 同步问题

- 共享内存允许多个进程同时访问，因此需要使用信号量或其他同步机制来确保数据的一致性。

2. 共享内存的大小

- 共享内存的大小由 `shmget()` 的第二个参数决定。

3. 共享内存的持久性

- 共享内存段在内核中存在，即使所有进程都退出，共享内存段仍然存在。
- 使用 `shmctl()` 删除共享内存段。

信号

1. 信号处理函数

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <signal.h>
4  #include <unistd.h>
5  #include <string.h>
6
7  // 信号处理函数
8  void signal_handler(int sig) {
9      if (sig == SIGUSR1) {
10         printf("Received SIGUSR1 signal\n");
11     } else if (sig == SIGUSR2) {
12         printf("Received SIGUSR2 signal\n");
13     }
14 }
```

2. 发送信号的进程

```
1  int main() {
2      pid_t pid;
3
4      // 获取接收进程的 PID
5      printf("Enter the PID of the receiving process: ");
6      scanf("%d", &pid);
7
8      // 发送 SIGUSR1 信号
9      if (kill(pid, SIGUSR1) == -1) {
10         perror("kill");
11     }
12 }
```

```

11     exit(EXIT_FAILURE);
12 }
13
14 // 发送 SIGUSR2 信号
15 if (kill(pid, SIGUSR2) == -1) {
16     perror("kill");
17     exit(EXIT_FAILURE);
18 }
19
20 return 0;
21 }

```

3. 接收信号的进程

```

1 int main() {
2     // 注册信号处理函数
3     signal(SIGUSR1, signal_handler);
4     signal(SIGUSR2, signal_handler);
5
6     // 无限循环等待信号
7     while (1) {
8         pause(); // 挂起进程，等待信号
9     }
10
11     return 0;
12 }

```

代码说明

1. 信号处理函数

- 使用 `signal()` 函数注册信号处理函数。
- 当指定的信号到达时，信号处理函数会被调用。

2. 发送信号

- 使用 `kill()` 函数发送信号。
- `kill()` 的第一个参数是目标进程的 PID，第二个参数是信号类型。

3. 接收信号

- 使用 `signal()` 注册信号处理函数。
- 使用 `pause()` 挂起进程，等待信号。

注意事项

1. 信号类型

- 常见的信号类型包括 `SIGINT`（中断）、`SIGTERM`（终止）、`SIGUSR1` 和 `SIGUSR2`（用户自定义信号）。

2. 信号的同步问题

- 信号是异步的，处理信号时需要特别注意同步问题。
- 在信号处理函数中尽量避免复杂的操作，只执行简单的任务。

3. 信号的阻塞和忽略

- 使用 `signal()` 可以忽略信号（设置处理函数为 `SIG_IGN`）。
- 使用 `sigprocmask()` 可以阻塞信号的传递。

4. 信号的可靠性

- 信号可能丢失，尤其是在高负载情况下。
- 如果需要可靠的通知机制，可以考虑使用其他 IPC 机制（如消息队列）。

信号量

1. 信号量结构体

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <sys/ipc.h>
5  #include <sys/shm.h>
6  #include <sys/sem.h>
7  #include <unistd.h>
8
9  // 信号量结构体
10 union semun {
11     int val;
12     struct semid_ds *buf;
13     unsigned short *array;
14     struct seminfo *__buf;
15 };
```

2. 创建信号量

```
1  int main() {
2      key_t key;
3      int semid;
4
5      // 创建一个唯一的键
6      key = ftok("progfile", 65);
7      if (key == -1) {
8          perror("ftok");
9          exit(EXIT_FAILURE);
10     }
11
12     // 创建信号量
13     semid = semget(key, 1, 0666 | IPC_CREAT);
14     if (semid == -1) {
15         perror("semget");
16         exit(EXIT_FAILURE);
17     }
18
19     // 初始化信号量
20     union semun arg;
21     arg.val = 1; // 初始值为1
22     if (semctl(semid, 0, SETVAL, arg) == -1) {
23         perror("semctl");
24         exit(EXIT_FAILURE);
25     }
```



```
26
27     return semid;
28 }
```

3. 生产者进程

```
1  int main() {
2      key_t key;
3      int semid;
4      struct sembuf sem_op;
5
6      // 创建一个唯一的键
7      key = ftok("progfile", 65);
8      if (key == -1) {
9          perror("ftok");
10         exit(EXIT_FAILURE);
11     }
12
13     // 获取信号量
14     semid = semget(key, 1, 0666 | IPC_CREAT);
15     if (semid == -1) {
16         perror("semget");
17         exit(EXIT_FAILURE);
18     }
19
20     // 生产者逻辑
21     for (int i = 0; i < 5; i++) {
22         // 等待信号量
23         sem_op.sem_num = 0;
24         sem_op.sem_op = -1;
25         sem_op.sem_flg = 0;
26         if (semop(semid, &sem_op, 1) == -1) {
27             perror("semop");
28             exit(EXIT_FAILURE);
29         }
30
31         printf("Producer: Produced item %d\n", i + 1);
32
33         // 发布信号量
34         sem_op.sem_op = 1;
35         if (semop(semid, &sem_op, 1) == -1) {
36             perror("semop");
37             exit(EXIT_FAILURE);
38         }
39
40         sleep(1); // 模拟生产时间
41     }
42
43     return 0;
44 }
```

4. 消费者进程

```
1  int main() {
2      key_t key;
3      int semid;
4      struct sembuf sem_op;
5
6      // 创建一个唯一的键
7      key = ftok("progfile", 65);
8      if (key == -1) {
9          perror("ftok");
10         exit(EXIT_FAILURE);
11     }
12
13     // 获取信号量
14     semid = semget(key, 1, 0666 | IPC_CREAT);
15     if (semid == -1) {
16         perror("semget");
17         exit(EXIT_FAILURE);
18     }
19
20     // 消费者逻辑
21     for (int i = 0; i < 5; i++) {
22         // 等待信号量
23         sem_op.sem_num = 0;
24         sem_op.sem_op = -1;
25         sem_op.sem_flg = 0;
26         if (semop(semid, &sem_op, 1) == -1) {
27             perror("semop");
28             exit(EXIT_FAILURE);
29         }
30
31         printf("Consumer: Consumed item %d\n", i + 1);
32
33         // 发布信号量
34         sem_op.sem_op = 1;
35         if (semop(semid, &sem_op, 1) == -1) {
36             perror("semop");
37             exit(EXIT_FAILURE);
38         }
39
40         sleep(1); // 模拟消费时间
41     }
42
43     // 删除信号量
44     union semun arg;
45     if (semctl(semid, 0, IPC_RMID, arg) == -1) {
46         perror("semctl");
47         exit(EXIT_FAILURE);
48     }
49
50     return 0;
51 }
```

代码说明

1. 创建信号量

- 使用 `ftok()` 生成一个唯一的键。
- 使用 `semget()` 创建或获取信号量。
- 使用 `semctl()` 初始化信号量。

2. 生产者逻辑

- 使用 `semop()` 等待信号量 (`sem_op = -1`)。
- 模拟生产操作。
- 使用 `semop()` 发布信号量 (`sem_op = 1`)。

3. 消费者逻辑

- 使用 `semop()` 等待信号量 (`sem_op = -1`)。
- 模拟消费操作。
- 使用 `semop()` 发布信号量 (`sem_op = 1`)。
- 使用 `semctl()` 删除信号量。

注意事项

1. 信号量的初始化

- 信号量的初始值决定了资源的可用性。

2. 信号量的同步

- `semop()` 用于执行信号量操作，`sem_flg` 为 0 表示阻塞操作。

3. 信号量的持久性

- 信号量在内核中存在，即使所有进程都退出，信号量仍然存在。
- 使用 `semctl()` 删除信号量。

4. 错误处理

- 在实际应用中，需要仔细处理可能的错误，例如权限问题、信号量创建失败等。

Socket

1. 服务端代码

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <arpa/inet.h>
6
7  #define PORT 8080
8  #define MAX_CLIENTS 5
9  #define BUFFER_SIZE 1024
10
11 int main() {
12     int server_fd, new_socket;
13     struct sockaddr_in address;
14     int opt = 1;
```

```
15     int addrlen = sizeof(address);
16     char buffer[BUFFER_SIZE] = {0};
17
18     // 创建 socket
19     if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
20         perror("socket failed");
21         exit(EXIT_FAILURE);
22     }
23
24     // 设置 socket 选项
25     if (setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR | SO_REUSEPORT, &opt,
17 sizeof(opt))) {
26         perror("setsockopt");
27         exit(EXIT_FAILURE);
28     }
29
30     // 绑定地址
31     address.sin_family = AF_INET;
32     address.sin_addr.s_addr = INADDR_ANY;
33     address.sin_port = htons(PORT);
34     if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
35         perror("bind failed");
36         exit(EXIT_FAILURE);
37     }
38
39     // 监听
40     if (listen(server_fd, MAX_CLIENTS) < 0) {
41         perror("listen");
42         exit(EXIT_FAILURE);
43     }
44
45     printf("Server listening on port %d...\n", PORT);
46
47     // 接受连接
48     if ((new_socket = accept(server_fd, (struct sockaddr *)&address,
17 (socklen_t *)&addrlen)) < 0) {
49         perror("accept");
50         exit(EXIT_FAILURE);
51     }
52
53     printf("Client connected\n");
54
55     // 读取数据
56     read(new_socket, buffer, BUFFER_SIZE);
57     printf("Received from client: %s\n", buffer);
58
59     // 发送数据
60     const char *response = "Hello from server";
61     send(new_socket, response, strlen(response), 0);
62     printf("Message sent to client: %s\n", response);
63
64     // 关闭 socket
65     close(new_socket);
66     close(server_fd);
67
68     return 0;
```

2. 客户端代码

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <arpa/inet.h>
6
7  #define PORT 8080
8  #define BUFFER_SIZE 1024
9
10 int main() {
11     int sock = 0;
12     struct sockaddr_in serv_addr;
13     char buffer[BUFFER_SIZE] = {0};
14
15     // 创建 socket
16     if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
17         printf("\nSocket creation error\n");
18         return -1;
19     }
20
21     // 设置服务器地址
22     serv_addr.sin_family = AF_INET;
23     serv_addr.sin_port = htons(PORT);
24
25     // 将字符串形式的 IP 地址转换为网络字节序
26     if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
27         printf("\nInvalid address\n");
28         return -1;
29     }
30
31     // 连接到服务器
32     if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0)
33     {
34         printf("\nConnection failed\n");
35         return -1;
36     }
37     printf("Connected to server\n");
38
39     // 发送数据
40     const char *message = "Hello from client";
41     send(sock, message, strlen(message), 0);
42     printf("Message sent to server: %s\n", message);
43
44     // 接收数据
45     read(sock, buffer, BUFFER_SIZE);
46     printf("Received from server: %s\n", buffer);
47
48     // 关闭 socket
49     close(sock);
50 }
```

```
51     return 0;
52 }
```

代码说明

1. 服务端逻辑

- 创建 socket: `socket()`。
- 设置 socket 选项: `setsockopt()`。
- 绑定地址: `bind()`。
- 监听连接: `listen()`。
- 接受连接: `accept()`。
- 读取数据: `read()`。
- 发送数据: `send()`。
- 关闭 socket: `close()`。

2. 客户端逻辑

- 创建 socket: `socket()`。
- 设置服务器地址: `inet_pton()`。
- 连接到服务器: `connect()`。
- 发送数据: `send()`。
- 接收数据: `read()`。
- 关闭 socket: `close()`。

注意事项

1. 网络字节序

- 使用 `htons()` 和 `inet_pton()` 将主机字节序转换为网络字节序。

2. 错误处理

- 在实际应用中, 需要仔细处理可能的错误, 例如连接失败、数据传输失败等。

3. 多客户端支持

- 服务端可以使用多线程或多进程来支持多个客户端连接。

4. 超时处理

- 可以设置 socket 超时时间, 避免阻塞。

线程同步

- 1 # 互斥锁：使用互斥锁（**Mutex**）可以保证在同一时间只有一个线程可以访问共享资源。当一个线程获得了互斥锁后，其他线程需要等待该线程释放锁才能继续访问共享资源。
- 2
- 3 # 信号量：信号量（**Semaphore**）是一种计数器，用来控制同时访问某个共享资源的线程数量。当计数器大于0时，线程可以访问资源并将计数器减1；当计数器等于0时，线程需要等待其他线程释放资源后才能继续访问。
- 4
- 5 # 条件变量：条件变量（**Condition Variable**）用于线程之间的通信和协调。一个线程可以等待某个条件的发生，而另一个线程可以在满足条件时通知等待的线程继续执行。
- 6
- 7 # 读写锁：读写锁（**Read-Write Lock**）允许多个线程同时读取共享资源，但只允许一个线程写入共享资源。这样可以提高读取操作的并发性，同时保证写入操作的原子性。

互斥锁

1. 创建互斥锁

- 使用 `pthread_mutex_init()` 初始化互斥锁。

2. 锁定互斥锁

- 使用 `pthread_mutex_lock()` 锁定互斥锁。

3. 访问共享资源

- 在锁定互斥锁后，访问共享资源。

4. 解锁互斥锁

- 使用 `pthread_mutex_unlock()` 解锁互斥锁。

5. 销毁互斥锁

- 使用 `pthread_mutex_destroy()` 销毁互斥锁。

信号

信号量是一种计数器，用于控制对共享资源的访问。

实现过程

1. 创建信号量

- 使用 `sem_init()` 初始化信号量。

2. 等待信号量

- 使用 `sem_wait()` 等待信号量。

3. 发布信号量

- 使用 `sem_post()` 发布信号量。

4. 销毁信号量

- 使用 `sem_destroy()` 销毁信号量。

条件变量

条件变量用于在特定条件满足时通知线程。

实现过程

1. 创建条件变量

- 使用 `pthread_cond_init()` 初始化条件变量。

2. 等待条件变量

- 使用 `pthread_cond_wait()` 等待条件变量。

3. 通知条件变量

- 使用 `pthread_cond_signal()` 或 `pthread_cond_broadcast()` 通知条件变量。

4. 销毁条件变量

- 使用 `pthread_cond_destroy()` 销毁条件变量。

读写锁

读写锁允许多个读线程同时访问共享资源，但写线程需要独占访问。

实现过程

1. 创建读写锁

- 使用 `pthread_rwlock_init()` 初始化读写锁。

2. 读锁定

- 使用 `pthread_rwlock_rdlock()` 获取读锁。

3. 写锁定

- 使用 `pthread_rwlock_wrlock()` 获取写锁。

4. 解锁

- 使用 `pthread_rwlock_unlock()` 解锁。

5. 销毁读写锁

- 使用 `pthread_rwlock_destroy()` 销毁读写锁。

线程池

线程池是一种管理线程的机制，用于提高多线程程序的性能。

实现过程

1. 创建线程池

- 初始化线程池，创建一定数量的线程。

2. 任务队列

- 使用队列存储待处理的任务。

3. 任务分配

- 线程从队列中获取任务并执行。

4. 线程管理

- 管理线程的创建、销毁和任务分配。

并发并行 同步异步

- 1 并发：多个任务在同一个时间段内同时执行，如果是单核心计算机，CPU 会不断地切换任务来完成并发操作。
- 2
- 3 并行：多任务在同一个时刻同时执行，计算机需要有多核心，每个核心独立执行一个任务，多个任务同时执行，不需要切换。
- 4
- 5 同步：多任务开始执行，任务 A、B、C 全部执行完成后才算是结束。
- 6
- 7 异步：多任务开始执行，只需要主任务 A 执行完成就算结束，主任务执行的时候，可以同时执行异步任务 B、C，主任务 A 可以不需要等待异步任务 B、C 的结果。
- 8
- 9 并发、并行，是逻辑结构的设计模式。
- 10
- 11 同步、异步，是逻辑调用方式。
- 12
- 13 串行是同步的一种实现，就是没有并发，所有任务一个一个执行完成。
- 14
- 15 并发、并行是异步的 2 种实现方式。

软链接和硬链接

- 1 硬链接：
- 2 以文件副本的形式存在，所有的硬链接都指向同一个inode，他们都享有同一个inode 和一个数据块（data block）。但硬链接本身并不占用实际存储空间
- 3 一个文件（file）拥有多个文件名一样（file-硬链接1，file-硬链接2...）
- 4
- 5 软连接：
- 6 本身是一个单独的文件，拥有自己的inode 和自己的数据块。因此拥有自己的文件属性和权限
- 7
- 8 1、本质：
- 9 硬链接：同一个inode，只是多个名字。
- 10 软链接：是不同的文件，inode不同
- 11 2、跨分区
- 12 硬链接无法跨分区、跨设备建立，软链接可以
- 13 3、目录
- 14 硬链接无法创建目录硬链接，软链接可以
- 15 4、相互关系
- 16 硬链接没有主次之分，相互独立
- 17 软链接依赖于原文件，原文件被删除，软链接即不可用
- 18 5、链接数
- 19 硬链接会删除增加会影响链接数，软链接不会，因为inode不一样。
- 20 6、相对路径
- 21 硬链接创建时，原始文件路径是相对于当前路径。
- 22 软链接创建时，原始文件路径是 相对于软链接的路径
- 23 7、文件类型
- 24 硬链接的类型与原始文件类型一致，软链接则会显示 symbolic link
- 25 8、创建方式
- 26 硬链接创建：ln [原文件] [硬链接]
- 27 软链接创建：ln -s [原文件] [软链接]
- 28
- 29 硬链接的使用场景
- 30 当需要确保文件不会被意外删除时，可以使用硬链接。

31	硬链接不能跨文件系统，因此在同一个文件系统内使用。
32	软链接的使用场景
33	当需要跨文件系统或指向目录时，可以使用软链接。
34	软链接可以方便地指向不同的目标文件或目录。
35	权限问题
36	创建硬链接需要对目标文件的目录有写权限。
37	创建软链接需要对当前目录有写权限。

TCP UDP

1. TCP 的特性

- **面向连接**
 - 在数据传输之前，需要建立一个可靠的连接（三次握手）。
 - 数据传输完成后，需要关闭连接（四次挥手）。
- **可靠传输**
 - TCP 提供可靠的数据传输，确保数据包按顺序到达，并且没有丢失或损坏。
 - 使用确认（ACK）、重传和序列号机制来保证可靠性。
- **面向字节流**
 - TCP 将数据视为字节流，不保证消息边界。
- **高延迟**
 - 由于可靠性保证，TCP 的延迟较高。

2. UDP 的特性

- **无连接**
 - UDP 是无连接的，发送数据前不需要建立连接。
 - 每个数据包独立发送，没有连接的开销。
- **不可靠传输**
 - UDP 不保证数据包的可靠传输，可能会丢失或乱序。
 - 没有确认、重传和序列号机制。
- **面向数据报**
 - UDP 以数据报的形式发送数据，每个数据报独立传输，保证消息边界。
- **低延迟**
 - 由于没有可靠性保证，UDP 的延迟较低。

特性	TCP	UDP
	面向连接	无连接
	可靠传输	不可靠传输
	面向字节流	面向数据报
	高延迟	低延迟
	需要高可靠性的场景（如 HTTP、SMTP）	需要低延迟的场景（如 DNS、视频流）

2. UDP 客户端和服务端

服务端代码

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <arpa/inet.h>
6
7  #define PORT 8080
8  #define BUFFER_SIZE 1024
9
10 int main() {
11     int server_fd;
12     struct sockaddr_in address;
13     char buffer[BUFFER_SIZE] = {0};
14     socklen_t len;
15
16     // 创建 socket
17     if ((server_fd = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
18         perror("socket failed");
19         exit(EXIT_FAILURE);
20     }
21
22     // 绑定地址
23     address.sin_family = AF_INET;
24     address.sin_addr.s_addr = INADDR_ANY;
25     address.sin_port = htons(PORT);
26     if (bind(server_fd, (const struct sockaddr *)&address, sizeof(address))
27 < 0) {
28         perror("bind failed");
29         exit(EXIT_FAILURE);
30     }
31
32     printf("Server listening on port %d...\n", PORT);
33
34     // 接收数据
35     len = sizeof(address);
36     recvfrom(server_fd, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&address,
37 &len);
38     printf("Received from client: %s\n", buffer);
39
40     // 发送数据
41     const char *response = "Hello from UDP server";
42     sendto(server_fd, response, strlen(response), 0, (const struct sockaddr
43 *)&address, len);
44     printf("Message sent to client: %s\n", response);
45
46     // 关闭 socket
47     close(server_fd);
48
49     return 0;
50 }
```

客户端代码

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include <unistd.h>
5  #include <arpa/inet.h>
6
7  #define PORT 8080
8  #define BUFFER_SIZE 1024
9
10 int main() {
11     int sock = 0;
12     struct sockaddr_in serv_addr;
13     char buffer[BUFFER_SIZE] = {0};
14
15     // 创建 socket
16     if ((sock = socket(AF_INET, SOCK_DGRAM, 0)) < 0) {
17         printf("\nSocket creation error\n");
18         return -1;
19     }
20
21     // 设置服务器地址
22     serv_addr.sin_family = AF_INET;
23     serv_addr.sin_port = htons(PORT);
24
25     // 将字符串形式的 IP 地址转换为网络字节序
26     if (inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr) <= 0) {
27         printf("\nInvalid address\n");
28         return -1;
29     }
30
31     // 发送数据
32     const char *message = "Hello from UDP client";
33     sendto(sock, message, strlen(message), 0, (const struct sockaddr
34 *)&serv_addr, sizeof(serv_addr));
35     printf("Message sent to server: %s\n", message);
36
37     // 接收数据
38     socklen_t len = sizeof(serv_addr);
39     recvfrom(sock, buffer, BUFFER_SIZE, 0, (struct sockaddr *)&serv_addr,
40 &len);
41     printf("Received from server: %s\n", buffer);
42
43     // 关闭 socket
44     close(sock);
45
46     return 0;
47 }
```

1. 服务端

- 创建 socket。
- 绑定 socket 到特定的端口。
- 接收数据。

2. 客户端

- 创建 socket。
- 发送数据到服务端。
- 接收服务端的响应（如果需要）。

应用场景

• TCP 的应用场景

- 对数据完整性要求高的场景，如 HTTP、HTTPS、SMTP、FTP 等。
- 需要保证数据按顺序到达的场景。

• UDP 的应用场景

- 对实时性要求高的场景，如 DNS 查询、视频流、实时游戏等。
- 数据包丢失可以接受的场景。

- 1 # 我理解的三次握手和4次挥手，本质就是TCP使全双工连接，因此A请求B，B确认告诉A， B请求A，A确认告诉B，只是说开始的时候，B请求A由2步合为了一步。而4次挥手释放连接本质上和建立连接没啥区别，只是因为传送数据完毕之后A和B并不对等(可能A传输先完成，B传输未完成)因此中间的过程并不能像建立连接那样可以进行合并

递归

递归的基本原理

1. 递归调用

- 函数调用自身，每次调用时问题规模减小。

2. 终止条件 (Base Case)

- 递归必须有一个明确的终止条件，否则会导致无限递归，最终导致栈溢出。

3. 递归步骤 (Recursive Step)

- 将问题分解为更小的子问题，并调用自身来解决这些子问题。

递归的优缺点

• 优点

- 代码简洁，逻辑清晰。
- 适合解决分治问题（如排序、树的遍历等）。

• 缺点

- 可能导致栈溢出（如果递归深度过大）。
- 性能可能不如迭代（因为每次函数调用都有一定的开销）。

阶乘计算

```
1  #include <stdio.h>
2
3  int factorial(int n) {
4      if (n == 0) { // 终止条件
5          return 1;
6      }
7      return n * factorial(n - 1); // 递归调用
8  }
9
10 int main() {
11     printf("Factorial of 5: %d\n", factorial(5));
12     return 0;
13 }
```

new malloc

区别

语法和类型安全：*new* 是一个操作符，返回的是对象类型的指针，类型安全；*malloc* 是一个函数，返回 *void*，**需要强制类型转换**[12](#)。

内存分配失败处理：*new* 分配失败时会抛出 *bad_alloc* 异常；*malloc* 分配失败时返回 *NULL*[12](#)。

构造和析构：*new* 会调用构造函数初始化对象，*delete* 会调用析构函数；*malloc* 和 *free* 只分配和释放内存，不会调用构造和析构函数[12](#)。

内存区域：*new* 从自由存储区分配内存，*malloc* 从堆分配内存[3](#)。

重载：C++ 允许重载 *operator new* 和 *operator delete*，但不能重载 *malloc* 和 *free*[3](#)。

X协议

认识X11

- 1 X11是X协议，版本号为11。X协议是专门被设计为linux桌面管理服务的，而linux桌面环境不像windows那样作为系统内核的一部分，作为一个普通程序运行在用户态上。该协议的设计初衷是为了linux的图形界面满足跨平台、跨网络、与具体硬件剥离、同时被多个用户使用的需求，因此该协议被设计成客户端-服务器的模式，即由xserver和xclient组成，xserver和xclient通过网络架构来进行图形接口的通信和执行绘制：
- 2
- 3 xserver管理主机上与显示相关的硬件设置（如显卡、硬盘、鼠标等），它负责屏幕画面的绘制与显示，以及将输入设置（如键盘、鼠标）的动作告知 xclient。
- 4 xclient即linux主机，负责处理逻辑。
- 5 比如如果用户点击了鼠标左键，xserver 捕捉到鼠标点击这个动作，然后它把这个动作告诉 xclient，然后xclient 就根据程序预先设定的逻辑（例如画一个圆），告诉 x server说："请在屏幕xx位置（鼠标左键点击的位置）画一个圆"。最后，xserver 就响应 xclient 的请求，在鼠标点击的位置，绘制并显示出一个圆

x协议的实现

- 1 目前基于该协议的实现架构如下图所示，`xfree64`是`xserver`的具体实现，`xorg`是`xfree64`的扩展；`xlib`是`xclient`的具体实现，但是对开发者不够简便，于是在`xlib`上又诞生了`qt`和`gtk`，`qt`是直接使用`xlib`库的，`gtk`则使用了再次封装了的`gdk`和`glib`；`qt`与`gtk`都很优秀，在他们之上开发者分别开发出了`kde`与`gnome`桌面；`kde`和`gnome`是集成了窗口管理器及一些应用程序的套件；其他的`x application`则跑在这些窗口管理器里。

显示管理器(Display Manager)和桌面环境(也叫窗口管理器Window Manager)

- 1 显示管理器(Display Manager,DM) 主要用于提供linux登录环境和登录后加载桌面环境，我所理解的管理登录显示和选择桌面显示环境；**桌面环境(Window Manager,WM, 也就是X服务)**主要用来统一管控多个`xclient`的绘制，因为`xclient`之间彼此是不知道对方在屏幕上的信息（想想多个`xclient`都往同一个显示器上绘制，这样容易导致相互覆盖）。
- 2
- 3 桌面环境的作用还包括：
- 4
- 5 提供许多的控制元素，包括工作列、背景桌面的设置等等；
- 6 管理虚拟桌面（virtual desktop）；
- 7 提供窗口控制参数，这包括窗口的大小、窗口的重叠显示、窗口的移动、窗口的最小化等等。
- 8 要注意显示管理器和桌面环境是两个分开独立的应用程序，但是往往在同一个团队一起开发，用户可以通过显示管理器切换任意的桌面环境，比如目前常见的配套如下（左为显示管理器，右为桌面环境）：
- 9
- 10 GDM-----GNOME（GNU Network Object Model Environment，使用大量的系统资源，但给一个现代的、精致的系统）
- 11 KDM-----KDE（K Desktop Environment，可高度定制的桌面，适度占用系统资源）
- 12 LightDM(Light Display Manager, 由 Ubuntu 为 Unity 桌面开发，作为跨桌面显示管理器，其目的是成为 X.org 的 X Server 的标准显示管理器)
- 13 -----twm（Tab Window Manager）
- 14 -----XFCE（XForms Common Environment，外观复古但占用资源很少）
- 15 -----LXDE（唯一的重点是尽可能少地使用资源）
- 16 -----Budgie（现代的外观和适度占用系统资源）